

Brain Finance

Agent Configuration Schema & Implementation Guide

Complete user input mapping for AI agent deployment on Brain Finance

Document Version: 1.0 | **Status:** Ready for Replit Implementation | **Last Updated:** April 2026

Executive Summary

This document provides the complete user input schema for deploying AI agents on Brain Finance, mapped to on-chain enforcement via smart contracts. It covers all six agent types (Trading, Lending, Yield, Payments, Analytics, Custom), with data structures, policy enforcement layers, and API specifications.

Overview

Every agent deployed on Brain must have a validated configuration that flows through a pipeline: User Input (UI Form) → Control Plane / Policy Engine (off-chain validation) → Policy Hash Derivation (deterministic) → Signed Policy Proof (cryptographic commitment) → UserOperation Assembly → ERC-4337 EntryPoint → On-Chain Verification (PolicyValidator + Crossmint Agent Wallet) → Execution.

Key Design Principles:

- All execution is gated by a cryptographic proof derived from user inputs
- Policy enforcement happens at three independent layers: off-chain (PolicyEngine), on-chain (PolicyValidator), and smart contract guardrails (Crossmint Agent Wallet)
- A hallucinating LLM, compromised agent, or malicious template cannot bypass controls
- Full audit trail via ERC-8004 registry and receipt indexer
- Agent memory (pgvector + database) feeds observations back to Claude ReAct loop

Core Agent Inputs

Every agent requires these baseline fields: agent_name (max 64 chars), agent_description (max 256 chars), owner_address (inherited from Crossmint Embedded Wallet), agent_type (enum: trading | lending | yield | payments | analytics | custom), status (enum: draft | deployed | paused | retired), capital_allocation (USDC wei), created_at (timestamp), version (uint8 for policy upgrades).

Storage Locations: BrainAccount smart contract (ERC-4337 account), ERC-8004 agent registry (metadata and audit trail), Control Plane database (policy parameters)

Agent Type Schemas

1. Trading Agent

Purpose: Autonomous execution of perpetual and spot trades on Hyperliquid, subject to position limits and market constraints.

User Input Form includes: Strategy type (perpetual_long_short, grid_trading, yield_farming_arb, index_tracking, custom), Position controls (max_position_size_usdc, max_daily_loss_percent, allowed_markets, cooldown_window_seconds, cumulative_exposure_limit), Trading parameters (order_types, max_slippage_bps, max_position_leverage, rebalance_frequency_hours), Capital allocation (USDC in wei).

Enforcement Layers:

Tier 1 - PolicyEngine (Control Plane): Position size, market allowlist, velocity check, daily loss tracking

Tier 2 - PolicyValidator (On-chain): Proof verification, replay prevention, nonce/expiry check

Tier 3 - Crossmint Agent Wallet: Smart contract guardrails, spend limits, recipient restrictions

2. Lending Agent

Purpose: Supply capital to lending protocols and manage collateral positions under LTV constraints.

User Input Form includes: Protocol (aave, compound, morpho, custom_contract), Supply controls (max_supply_usd, allowed_collateral_assets, allowed_borrow_assets), Risk controls (max_ltv_percent, target_ltv_percent, rebalance_threshold_percent, max_liquidation_risk_percent, max_protocol_exposure_percent), Yield preferences (preferred_assets, min_apy_target_percent).

Enforcement: Tier 1 - Supply limits, collateral allowlist, LTV simulation. Tier 2 - Oracle-gated LTV verification. Tier 3 - Enforces cumulative spend limit, prevents non-whitelisted pool addresses.

3. Yield Agent

Purpose: Optimize stablecoin yield via liquidity provision, stable swaps, perpetual funding, or curve strategies.

User Input Form includes: Strategy type (stable_farming, lp_on_dex, perpetual_funding, curve_convex, custom), Yield targets (min_apy_percent, target_apy_percent, exit_if_apy_below_percent), Risk controls (max_slippage_bps, impermanent_loss_tolerance_percent, max_stable_pair_concentration, rebalance_frequency_hours, max_position_size_usdc).

Enforcement: Tier 1 - Validates entry slippage, position size, min APY, simulates IL impact. Tier 2 - Oracle-gated APY confirmation. Tier 3 - Enforces position limits and concentration caps.

4. Payments Agent

Purpose: Automate recurring bills, batch payments, and subscription management via x402 and direct USDC transfers.

User Input Form includes: Payment type (recurring_bills, direct_transfers, batch_payroll, subscription_manager, custom), Recipient configuration (allowlisted_recipients with per-payment limits and recurrence intervals, deny_list), Payment controls (per_transaction_limit_usdc, daily_spend_budget_usdc, daily_transaction_count_limit, require_approval_above_usdc, execution_window), x402 configuration (accept_x402_payments, x402_allowlist, x402_max_per_request_usdc).

Enforcement: Tier 1 - Recipient validation, budget/limit checks, daily transaction count, approval routing, x402 header parsing. Tier 2 - Proof verification with nonce/expiry. Tier 3 - Recipient whitelisting at smart contract level.

5. Analytics Agent

Purpose: Monitor portfolio state, detect anomalies, trigger alerts, and recommend rebalances. Read-only by default.

User Input Form includes: Monitoring scope (tracked_agents as array or 'all', tracked_positions), Alert rules (rule_name, condition, threshold, action, notification_target), Reporting (report_frequency, report_metrics, include_recommendations), Execution triggers (allow_auto_execute, execution_whitelist, execution_limit_usdc).

Enforcement: No budget enforcement for read-only operations. For auto-execute: PolicyEngine validates against execution_limit_usdc. ERC-8004 registry records all observations. pgvector stores historical data for trend analysis.

6. Custom Agent

Purpose: Deploy arbitrary agent objectives with user-defined execution boundaries.

User Input Form includes: Objective (free-form description, target_outcome, complexity_level), Execution framework (allowed_tools, forbidden_tools, custom_contract_calls with max_calls_per_day), Control parameters (primary_limit_usdc, secondary_limit, allowed_counterparties, execution_window, max_operations_per_hour, circuit_breaker_loss_percent), Integrations (webhook_for_notifications, custom_auth_headers, external_data_sources).

Enforcement: Flexible but bounded - tool whitelisting, forbidden tool blacklisting, operation rate limiting, cumulative P&L; circuit breakers. All executions go through standard PolicyValidator flow.

Data Flow

User Input (UI Form) flows through: Control Plane / Policy Engine (off-chain validation, constraint checking) → Policy Hash Derivation (deterministic keccak256) → Signed Policy Proof (cryptographic commitment with nonce & expiry) → UserOperation Assembly (attach proof, set gas) → Alchemy Bundler (validation & simulation) → ERC-4337 EntryPoint (forward to smart account) → PolicyValidator (verify proof, replay check) + Crossmint Agent Wallet (guardrails enforcement) → All Checks Pass? (YES: Execute action; NO: Revert & Error) → Receipt Indexer (record tx hash, agent_id, amount) → pgvector Memory (store observations, update history) → Claude ReAct Loop (observe outcome, update state, decide next action)

On-Chain Contract Interfaces

IBrainAccount (ERC-4337 Smart Account)

```
setPolicy(bytes32 policy_hash) - Update agent policy
getPolicy() returns bytes32 - Retrieve current policy
authorizeAgent(bytes8 agent_id, uint256 capital_allocation) - Deploy agent with capital
revokeAgent(bytes8 agent_id) - Deactivate agent
getAgentSubAccount(bytes8 agent_id) returns uint256 balance - Check agent balance
executeUserOp(UserOperation calldata userOp, bytes calldata policy_proof) - Execute
verified transaction
```

IPolicyValidator

```
verifyPolicy(bytes32 policy_hash, bytes calldata proof, bytes calldata input) returns bool
validateTradeIntent(TradeIntent calldata intent) returns bool
validatePaymentIntent(PaymentIntent calldata intent) returns bool
```

IERC8004 (Agent Registry)

```
registerAgent(bytes8 agent_id, address owner, string calldata agent_type, bytes calldata
metadata)
recordValidation(bytes8 agent_id, bytes32 validation_event_hash)
getAgentMetadata(bytes8 agent_id) returns bytes memory
```

Implementation Checklist

Phase 1: Core Inputs & Policy Compilation

UI form generation for all six agent types | Input validation (ranges, formats, ENS resolution) | Policy hash derivation (keccak256 hashing) | Deterministic test vectors | Signed proof generation | Proof verification utilities

Phase 2: On-Chain Enforcement

PolicyValidator contract development | Crossmint Agent Wallet integration | ERC-8004 registry deployment | Receipt indexer service | Event logging and audit trail | Oracle integration

Phase 3: AI Integration

Tool registry mapper | ReAct loop orchestration | Memory layer (pgvector + database schema) | System prompt generation per agent type | Error handling and rejection feedback

Phase 4: Demo Flows & Testing

Trading agent: position entry and exit | Lending agent: supply, borrow, LTV rebalance | Yield agent: LP entry and APY monitoring | Payments agent: recurring bills and x402 | Analytics agent: portfolio monitoring | Custom agent: user-defined execution | End-to-end integration testing

API Schemas

Create Agent

POST /api/agents/create

Request: agent_type, inputs (config), capital_allocation

Response: agent_id, policy_hash, status, tx_hash, capital_allocated, created_at

Get Agent Status

GET /api/agents/:agent_id

Response: agent_id, agent_type, status, owner_address, balance_usdc, policy_hash, recent_actions

Update Policy

PATCH /api/agents/:agent_id/policy

Request: policy_updates (object with changed fields)

Response: old_policy_hash, new_policy_hash, tx_hash, effective_at, status

Security Considerations

Input Validation: All numeric inputs validated for min/max, overflow/underflow

Nonce Management: Every proof includes nonce to prevent replay attacks

Expiry Enforcement: Proofs expire after set window (e.g., 10 minutes) to limit stale execution

Multi-Layer Defense: PolicyEngine (off-chain) + PolicyValidator (on-chain) + Crossmint guardrails (smart contract)

Audit Trail: Every action recorded in ERC-8004 registry + pgvector for full traceability

Whitelisting: Recipient and protocol allowlists enforced both off-chain and on-chain

Circuit Breakers: P&L; and volatility thresholds trigger halts, not auto-recovery

Next Steps

1. Set up Replit project with this schema
2. Build UI forms for agent configuration
3. Implement PolicyEngine for proof generation
4. Deploy PolicyValidator and ERC-8004 registry
5. Integrate Claude API with ReAct loop and tools
6. Connect Crossmint wallets for agent execution
7. Test end-to-end flows for each agent type

End of Document